Centro Universitário de Excelência
Sistemas de Informação

MecâniQA API — OAT 2

Ordens de Serviço, Padrão Builder e DTOs

Autor: Mateus Dantas de Morais
Levi Ramos Barbosa Santos
Ana Luiza Lima Nicolodi
Alisson Silva Nascimento
Julio Cesar Brito da Silva

Feira de Santana, 2026

Agenda

O objetivo desta apresentação é apresentar a evolução da API MecâniQA na OAT 2: novas entidades, o padrão Builder, a camada de DTOs e os testes da API.

1. Contexto
Regras de negócio e restrições da OAT 2

2. Entidades
OS, Pedido e a entidade associativa ItemPedido

3. Builder
Construção segura da Ordem de Serviço

4. DTOs e rotas
Mappers, endpoints e User Stories

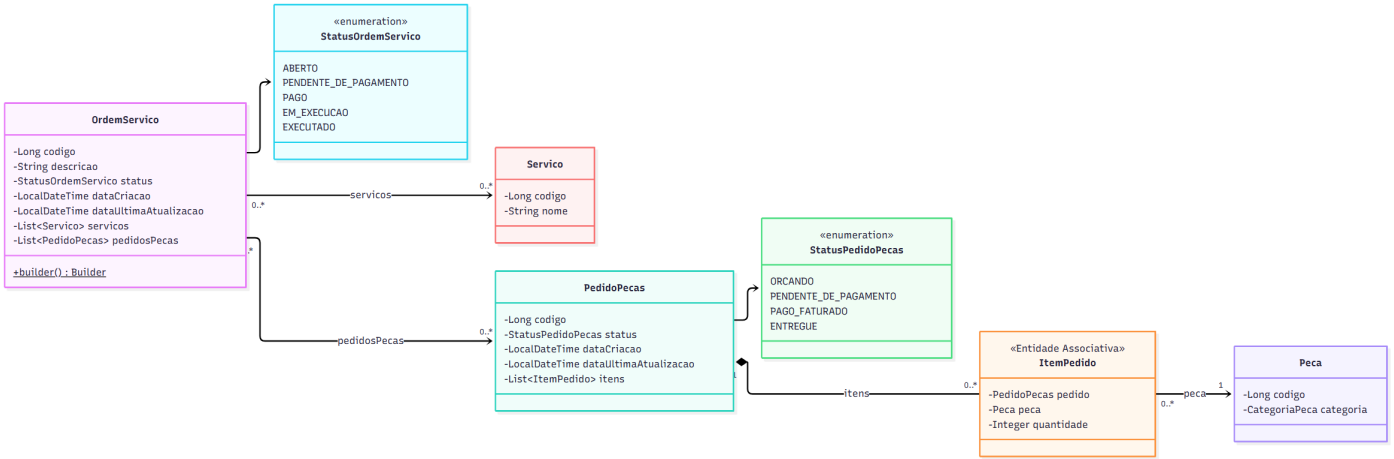
5. Testes e fluxo
QA, collection Postman e diagrama de atividade

Contexto e Regras de Negócio

- **OAT 2 evolui o CRUD de Peças e Serviços (OAT 1) com** Ordens de Serviço (OS) e Pedidos de Peças
- **Status obrigatórios da OS: Aberto, Pendente de Pagamento,** Pago, Em Execução e Executado
- **Status obrigatórios do Pedido de Peças: Orçando, Pendente** de Pagamento, Pago/Faturado e Entregue
- **Restrição tecnológica: uso da biblioteca Lombok proibido** construtores, getters e setters escritos manualmente
- **Persistência inalterada: repositórios Singleton estáticos,** somente em memória, sem banco de dados

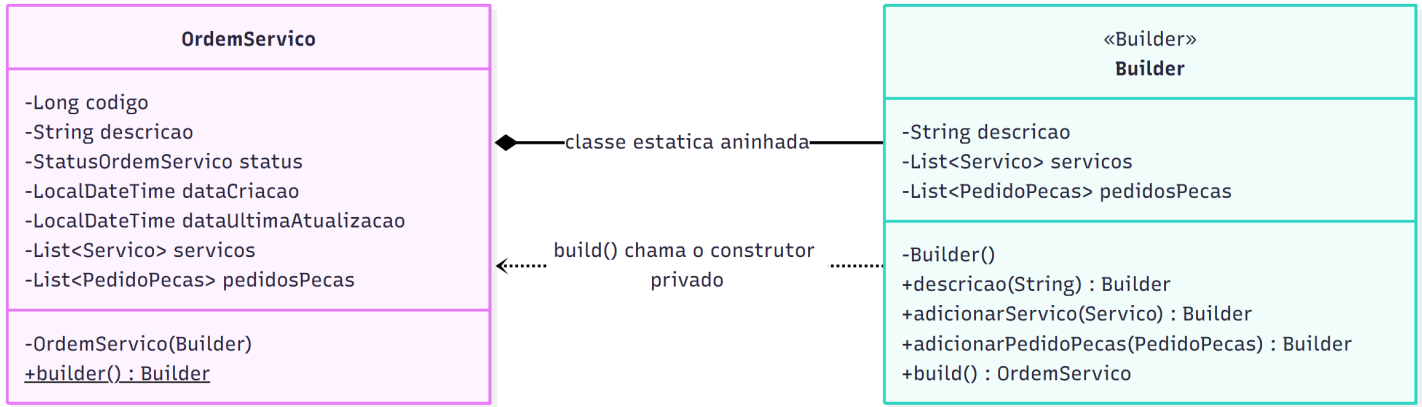
Novas Entidades e Entidade Associativa

OrdemServico e PedidoPecas seguem o modelo de Peça/Serviço
ItemPedido é a entidade associativa entre PedidoPecas e Peça,
guardando a quantidade de cada peça no pedido
Repetir a mesma peça em um pedido soma a quantidade
no item existente, em vez de duplicá-lo



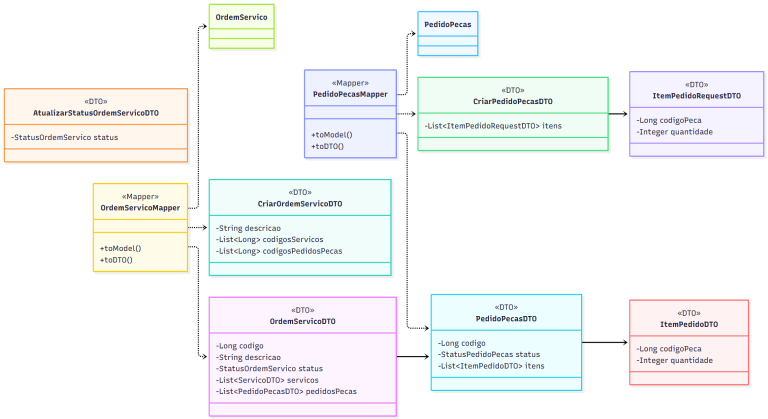
Padrão Builder na Ordem de Serviço

Construtor de OrdemServico é privado e só aceita um Builder
Builder é uma classe estática aninhada, com métodos que
retornam this e permitem encadeamento (fluent API)
build() é o único caminho até o construtor privado
Cenário de QA: new OrdemServico() não compila — só
OrdemServico.builder() constrói a entidade



DTOs e Mappers

Controllers não recebem nem devolvem entidades de domínio
Cada operação tem seu DTO: criação, atualização de status
e representação completa da resposta
Mappers convertem DTO <-> modelo nos dois sentidos
@JsonProperty(READ_ONLY) protege código e datas mesmo
se o cliente enviá-los no corpo da requisição



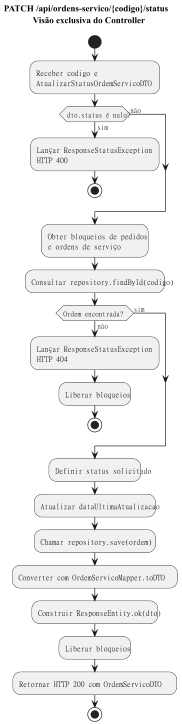
Endpoints e User Stories

US01 — Criar OS: POST /api/ordens-servico -> 201
US02 — Alterar status da OS: PATCH /ordens-servico/{codigo}/status -> 200
US03 — Criar pedido de peças: POST /api/pedidos-pecas -> 201
US04 — Adicionar peça ao pedido: POST /pedidos-pecas/{codigo}/itens -> 200
US05 — Alterar status do pedido: PATCH /pedidos-pecas/{codigo}/status -> 200
Refinamento: o brainstorm inicial previa PUT nas alterações de
status; a implementação usou PATCH, mais aderente a uma atualização parcial do recurso

Testes e Qualidade

- **Oat2IntegrationTest cobre as 5 User Stories, transições de** status, validações de erro e o padrão Builder
- **QaCamposInternosTest garante que o payload mínimo — sem** código, datas ou status — não quebra a API
- **Regra validada: repetir uma peça no mesmo pedido soma a** quantidade em vez de duplicar o item
- **Collection Postman (MecaniQA_API_OAT2) populada com os** cenários de sucesso e erro, exportada no repositório

Diagrama de Atividade do Controller



OrdemServicoController.modificarStatus — PATCH /api/ordens-servico/{codigo}/status

Considerações Finais

Builder isolou a complexidade de montar uma OS com serviços e pedidos associados, sem construtor público

A ausência do Lombok tornou o boilerplate visível e reforçou o entendimento de construtores, getters e setters

DTOs e Mappers desacoplaram o contrato HTTP do modelo de domínio, sem expor entidades internas ao cliente

Repositórios seguem em memória (Singleton estático) — os dados são perdidos a cada reinício da aplicação

Referências

Repositório: github.com/Jczar801/mecaniQA-api-Teresina

Enunciado da OAT 2 — Centro Universitário de Excelência,

curso de Sistemas de Informação

GAMMA, HELM, JOHNSON, VLISSIDES — Design Patterns:

Elements of Reusable Object-Oriented Software (padrão Builder)

Documentação oficial do Spring Boot 3.5 (spring.io/projects/spring-boot)